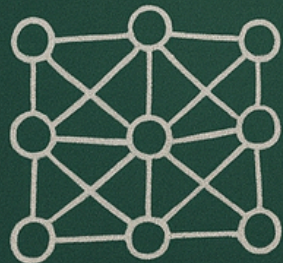
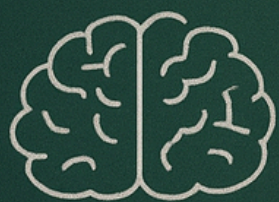


AI4Math

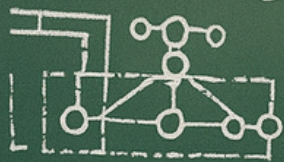
讨论班 03

模型结构-上



$f(x)$
ML

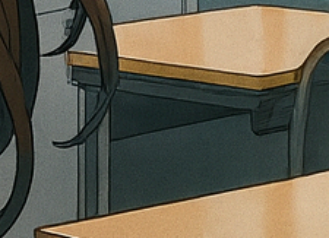
$$\frac{i\sqrt{x}}{\sqrt{x+1+0}}$$



$f(x)$
01



$0\sqrt{1}$



模型结构-上

AI4Math 讨论班第二季-03

晚星

求真书院

2025-10-25

前言

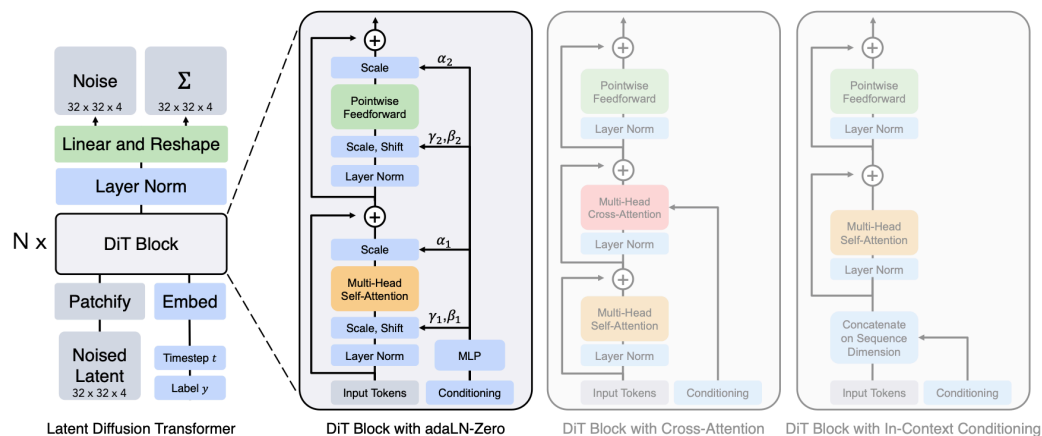
OpenAI Sora 2 视频生成模型发布于 2025 年 9 月 30 日，其在前代的基础上实现了物理一致性、动作幅度、多样性等等维度的大幅进步。Sora 2 同时提供免费版本给各家使用，由此开启了一个[抽象创作的时代](#)。

OpenAI 基本没有透露 Sora 2 实现方式的各种细节，不过根据此前的信息推测，其模型结构应该仍然是 DiT (Diffusion Transformer)。



Sora 初代于 2024 年 2 月首次公布，由此引起了视频生成模型的浪潮。(尽管它直到 2024 年底才正式公开，并且效果远不如早期宣传或者同期的其他视频生成模型)

Sora 所使用的模型结构来自 2023 年 3 月的一篇论文，*Scalable Diffusion Models with Transformers*。这篇论文的主要贡献在于使用标准的 transformer 架构替换了此前图像生成领域最常用的 U-Net 架构，并取得了不错的效果。它一开始被质疑创新性不足并被 CVPR 拒稿。

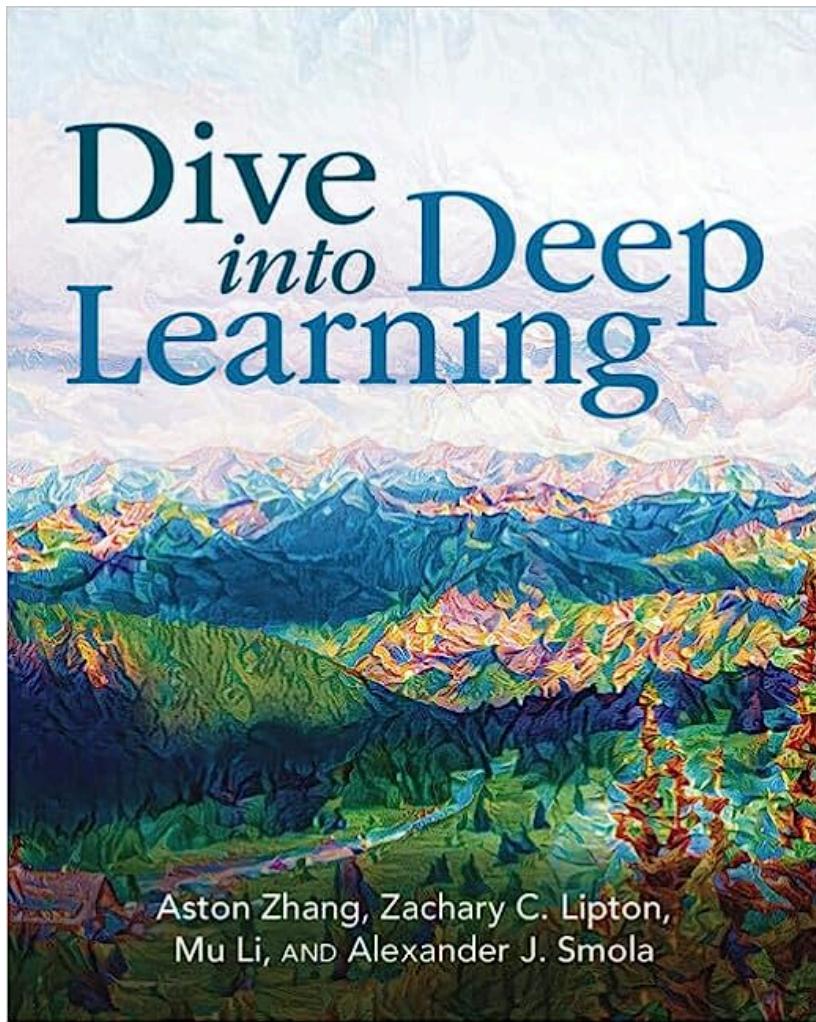


后续科普部分主要使用的参考书目是:

[Dive into Deep Learning](#)

这本书主要由李沐等一众 Amazon 大佬牵头编写，质量较高、简单易学且配有丰富的代码示例。也非常推荐用于自学使用。

不过书中内容基本截止于 2022 年，ChatGPT 流行之前的时间段。后续内容在我们的讨论班当中也会做一些补充。



ImageNet

在 2012 年，来自多伦多大学的三位研究者提出了一个名为「AlexNet」的深度学习神经网络，赢得了 2012 年大规模视觉识别挑战赛 ImageNet 的冠军。

三位作者如今都是 AI 领域里响当当的人物。Geoffrey Hinton 被誉为「深度学习之父」，Ilya Sutskever 是 OpenAI 的联合创始人及前首席科学家，Alex 同时也是 CIFAR-10 和 CIFAR-100 数据集的创建者。

在描述当年的 AlexNet 项目时，Geoffrey Hinton 总结道：「Ilya 认为我们应该做这件事，Alex 让它成功了，而我获得了诺贝尔奖。」

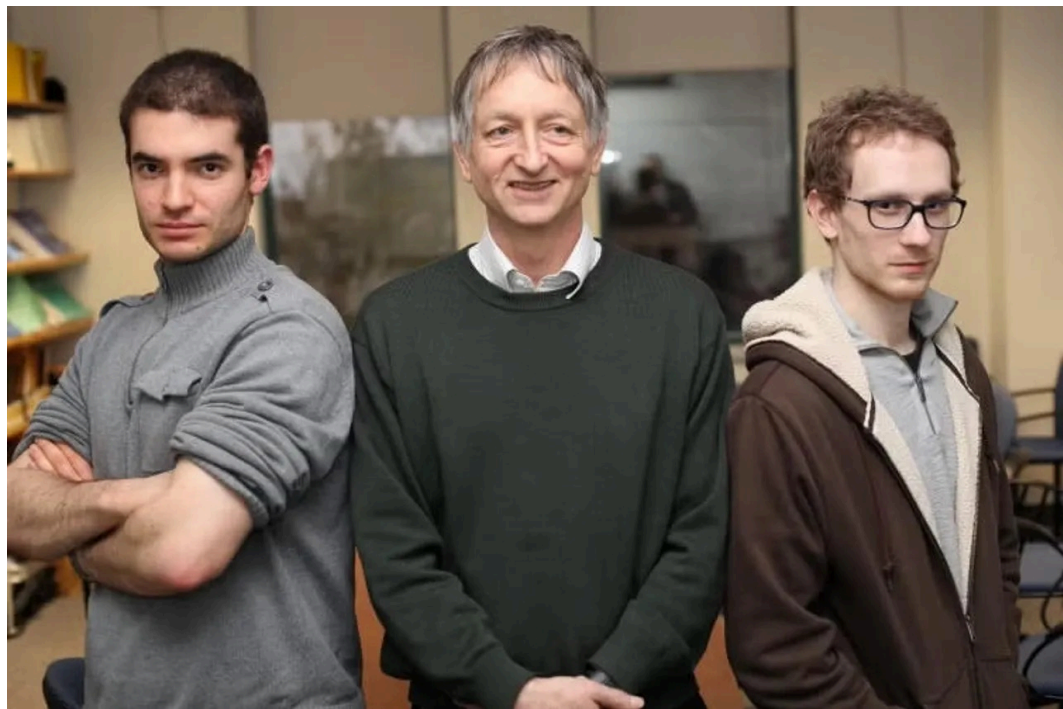


Figure 1: AlexNet 三位作者合影

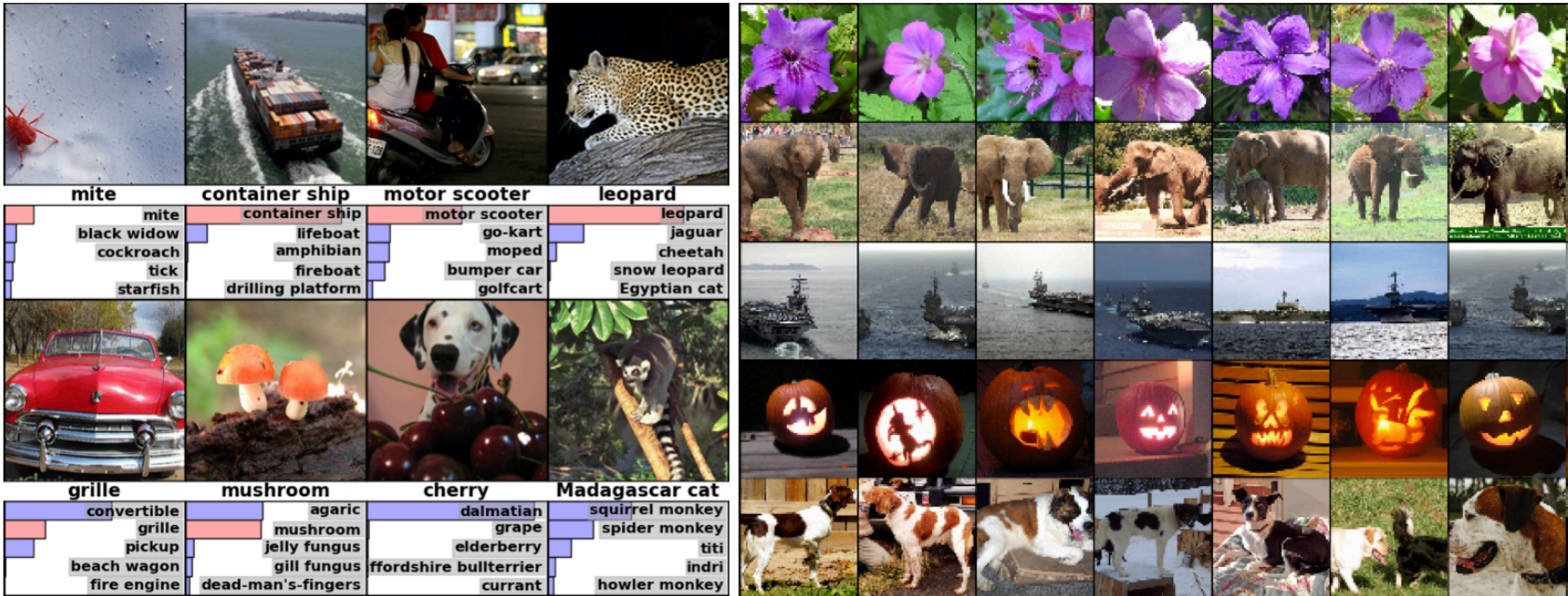
ImageNet 最早由斯坦福大学李飞飞团队带头，调动全球 167 个国家近五万名标注者在网络上实现众包标注，耗时三年完成了 1500 万张图像以及 22000 个类别的标注工作，成为了当时世界上最大的视觉数据集。ImageNet 数据集最早于 2009 年投递到了计算机视觉顶会 CVPR 当中，并作为 poster 论文中稿，受到的关注并不多。

后来为了增强 ImageNet 的影响力，李飞飞团队举办了 ImageNet 大规模视觉识别挑战赛，需要参赛选手在一个 ImageNet 的子集 140 万张图像当中完成 1000 个类别的分类任务。



Figure 2: 李飞飞及她的自传宣传图

在开始的两年里 (2010 - 2011), ImageNet 赛事并没有收获预期的效果, 参赛选手基本上都直接使用支持向量机算法参赛, 效果进步有限且没有带来什么新的技术突破。因此 ImageNet 早期参赛人数逐年下降, 逐渐遇冷。直到 AlexNet 之后取得了显著的性能提升, 这一赛事才再次受到关注。



经典神经网络

MNIST 是在 1998 年由 Yann LeCun 发布的经典图像识别测试集，主要测试的是手写数字的识别任务。整个 MNIST 由 6 万张训练集和 1 万张测试集组成，每一个图像都是 28x28 分辨率的灰度图像，共分为数字 0-9 十个类别。

我们通常采用 one-hot 编码以及概率观点来处理图像分类问题。具体而言我们期望的图像分类器是一个函数 $\varphi: \mathbb{R}^{28 \times 28} \rightarrow \mathbb{R}^{10}$ ，输入图像后直接输出图像属于每一个类别的概率。在深度学习当中我们就可以通过训练一个神经网络来拟合构造这一函数。



Figure 3: MNIST 图像分类数据集示例

为实现这一目标最简单的就是使用线性神经网络。我们可以直接通过将图像展平成一个 28×28 尺寸的向量，然后引入一个形状为 $M(28 \times 28, 10)$ 的参数矩阵并使用矩阵乘法计算得到结果，如有需要也可以额外引入一个偏置等等。最终我们通过一个softmax 函数就可以将输出数值固定转换为一个概率分布。

$$\mathbf{O} = \mathbf{XW} + \mathbf{b},$$

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{O})$$

在这其中 $\text{softmax} : \mathbb{R}^n \rightarrow [0, 1]^n, \mathbf{o} \mapsto \mathbf{y}$ where $y_i = (\exp(o_i)) / \left(\sum_j \exp(o_j) \right)$ 。

```
# 数据加载
transform =
transforms.Compose([transforms.ToTensor()])
train_data = datasets.MNIST('./data',
train=True, download=True,
transform=transform)
test_data = datasets.MNIST('./data',
train=False, transform=transform)
train_loader = DataLoader(train_data,
batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_data,
batch_size=batch_size)

# 定义模型（单层线性网络）
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28*28, 10)
)
```


交叉熵 (Cross Entropy) 的概念来自于信息论，对于概率分布 p, q 而言，一个概率分布的熵就是

$$H(p) = - \int \log p(x) dF_p(x)$$

它用于衡量一个随机变量的不确定性。同时两个随机变量的交叉熵就是

$$H(p, q) = - \int \log q(x) dF_p(x)$$

交叉熵度量的是，如果使用分布 q 来近似分布 p ，我们所需要的额外信息。只有当 $p = q$ 时交叉熵才会取得最小值，刚好等于 $H(p)$ 。

构造损失函数和优化器

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(),
                        lr=learning_rate)
```

优化器上我们就选用最直接的 SGD (随机梯度下降) 优化器即可。我们将最小化模型与真实分布的交叉熵作为训练目标，其参数 θ 的更新规则就是简单的

$$\theta \leftarrow \theta - \eta \nabla \tilde{H}(p, q_\theta)$$

这里的 $\tilde{H}$ 是根据随机采样数据估算的交叉熵， q_θ 就是我们的神经网络。

完整训练代码参见 [GitHub 仓库](#) 当中的“src/linear_mnist.py”文件，在完成环境配置之后可以直接在家用电脑的 CPU 上快速运行。

在经过多轮训练之后，这样的神经网络实际上已经能在这个任务上达成 90% 的准确率了。(此时总参数量 7,840)

```
Epoch [1/5], 准确率: 86.68%  
Epoch [2/5], 准确率: 88.35%  
Epoch [3/5], 准确率: 89.18%  
Epoch [4/5], 准确率: 89.70%  
Epoch [5/5], 准确率: 90.01%
```

```
# 训练循环  
for epoch in range(epochs):  
    # 训练阶段  
    model.train()  
    print(f"开始第{epoch+1}轮训练")  
    for images, labels in  
tqdm(train_loader):  
        outputs = model(images)  
        loss = criterion(outputs, labels)  
  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
  
    # 测试阶段  
    model.eval()  
    # ...
```

这里我们同样可以实现一个多层感知机 (Multi-Layer Perceptron, MLP) 作为一个更强的模型来执行这一任务。只需要在数个线性层中间叠加上合适的激活函数，即可实现一个经典的多层感知机。

可以看出通常情况下更大的模型训练速度更慢，但最终能够达到更强的性能上限。
(总参数量 535,818)

```
Epoch [1/5], 准确率: 83.61%  
Epoch [2/5], 准确率: 89.04%  
Epoch [3/5], 准确率: 90.32%  
Epoch [5/5], 准确率: 91.98%  
Epoch [10/10], 准确率: 94.15%
```

具体实现可参考文件“src/mlp_mnist.py”，此处相比较前文仅仅修改了神经网络结构，关键代码为

```
model = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(28*28, 512),      # 第一层：  
    输入层→隐藏层1  
    nn.ReLU(),                 # 激活函数  
    nn.Linear(512, 256),       # 第二层：  
    隐藏层1→隐藏层2  
    nn.ReLU(),                 # 激活函数  
    nn.Linear(256, 10)         # 第三层：  
    隐藏层2→输出层  
)
```


线性神经网络和多层感知机本质上第一步都是将一个二维图像展平转换成一个一维的向量再进行处理，但是这样一来很显然会忽略掉图像这一模态当中的很多特性，最终效果就相对欠佳。此时如果能够利用上我们关于图像的一些先验认知，就有可能使用大幅提高神经网络在图像任务上的表现。

图像模态有两个重要的先验特性可以使用：

局部性 图像当中的物体特征更多依赖于局部的信息

平移不变性 同一个物体位置移动之后其内容不会改变

而我们所使用的线性神经网络恰恰是位置敏感的，在有限的参数量和训练数据当中难以有效地处理这两种特性。

在 1998 年时 Yann LeCun 首先做出了第一个深度卷积神经网络，并在 MNIST 任务上取得了极好的性能表现。

在数学上有一种运算方式可以很好的契合这些特性，这就是卷积运算 (Convolution)。一般在连续情形下卷积运算的定义是对于两个函数 $f, g : \mathbb{R}^d \rightarrow \mathbb{R}$ ，它们之间的卷积就是一个函数

$$(f * g)(x) = \int f(z)g(x - z) dz$$

通常来说计算机当中显示的图像都不是一个简单的二维矩阵，而是一个三阶张量的形式。除去后两个维度分别代表长宽之外，第一个维度通常是 RGB 或 RGBA 等不同的色彩通道。

在我们神经网络当中则基本上都使用离散的卷积运算，处理各个不同位置上像素之间的对应关系。我们可以直观地想象这种卷积运算，它实际上是原本输入每一个位置的局部信息经过卷积核的加权求和，如下图所示。

Input		Kernel		Output																	
<table><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

使用卷积运算的深度神经网络就被称为卷积神经网络 (Convolutional Neural Network, CNN)。

卷积层同样具有线性性质，因此需要使用 ReLU 等激活函数来防止坍缩。与此同时 CNN 也常常搭配各种池化层以及线性层来调整输出张量的形状。

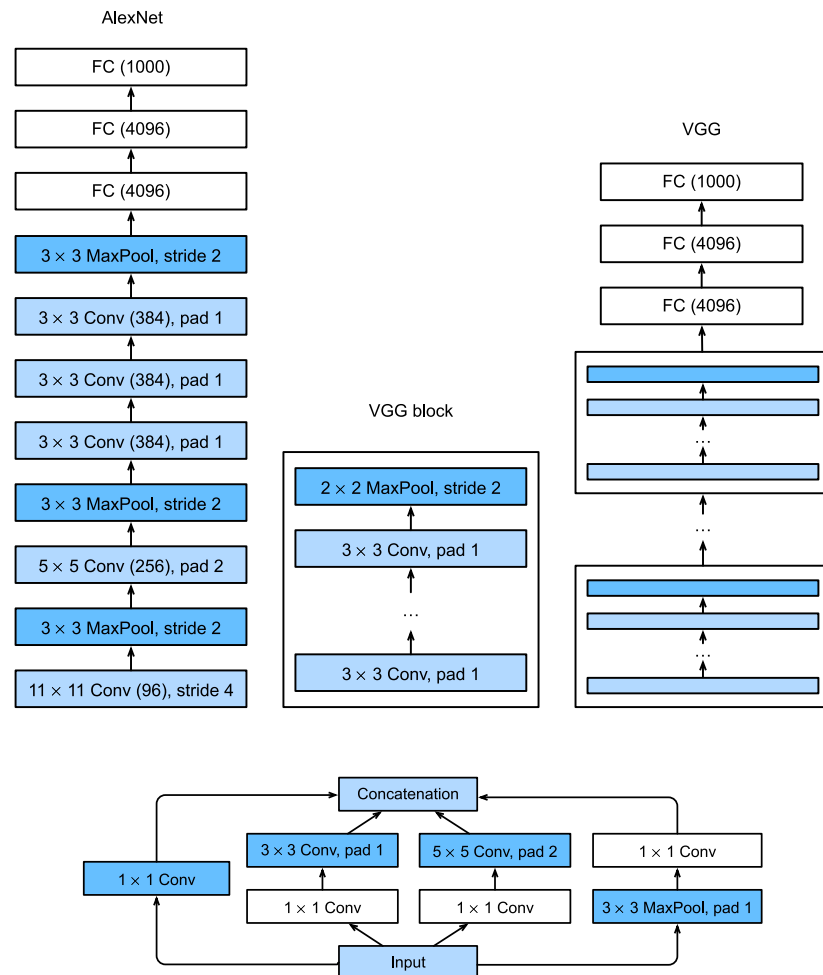
可以看到在训练后 CNN 在图像分类上的性能表现明显强出很多 (参见“cnn_mnist.py”), 而 2012 年震惊世界的 AlexNet 本质上也是一个多层的 CNN。(总参数量 421,642)

```
Epoch [1/5], 准确率: 90.59%  
Epoch [2/5], 准确率: 93.79%  
Epoch [3/5], 准确率: 95.26%  
Epoch [4/5], 准确率: 95.62%  
Epoch [5/5], 准确率: 96.72%
```

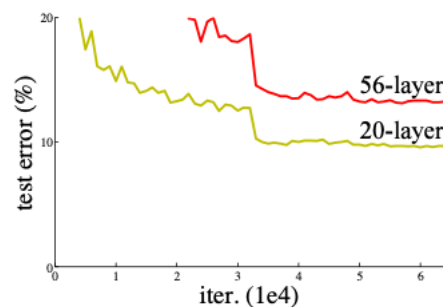
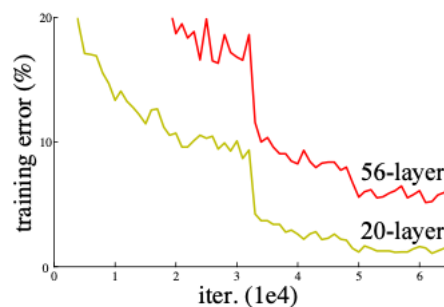
```
model = nn.Sequential(  
    # in: (28, 28), out: (32, 28, 28)  
    nn.Conv2d(1, 32, kernel_size=3,  
padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2), # 14x14  
    # in: (32, 14, 14), out: (64, 14, 14)  
    nn.Conv2d(32, 64, kernel_size=3,  
padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2), # 7x7  
    # in: (64, 7, 7), out: (64 * 7 * 7)  
    nn.Flatten(),  
    nn.Linear(64*7*7, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10)  
)
```

从 2012 年到 2014 年三届 ImageNet 竞赛当中获胜的模型基本上都采用了这类 CNN 模型结构。其中比较有代表性的进展有

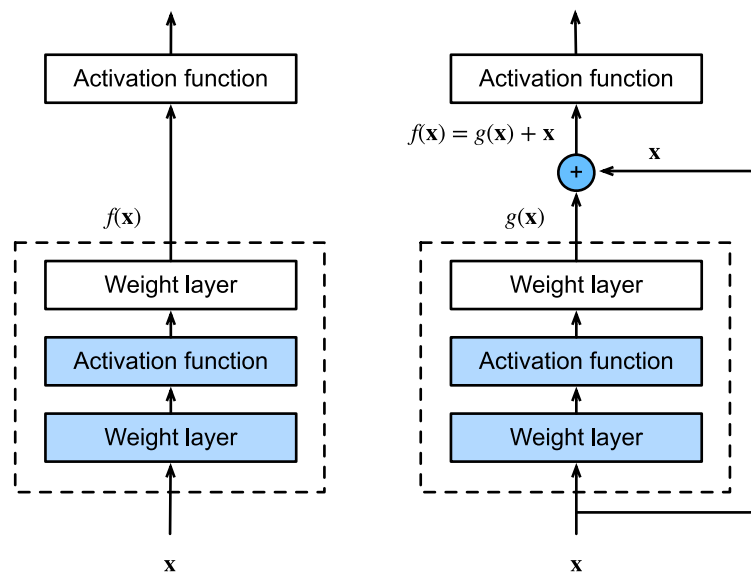
- AlexNet: 第一个使用 GPU 训练的 CNN, 效果拔群
- VGG: 使用单纯重复简单的 3x3 卷积层取得效果, 并在简单的模型结构下观察到了扩展潜力
- Inception: 结合使用多尺度卷积分支提取信息, 取得了更好的性能表现



时间来到 2015 年，这时人们发现了一种灵异现象，随着 VGG 类卷积神经网络深度以及参数量的扩大，模型的性能表现居然最终呈现出了下降的趋势，即使训练集上也同样如此。



此时最关键的突破来自于 2016 年何凯明的残差神经网络 (Residual Network)，它通过创新性的残差连接结构基本确保了随着模型结构的扩展神经网络的性能总可以持续提升下去。时至今日几乎所有的深度神经网络都会采用残差连接这一结构。ResNet 也成为整个 21 世纪引用量最高的一篇论文。



ResNet 最初的灵感来自于函数系，如 Figure 4 所示。如果我们把一个参数待定的神经网络看作能够覆盖一个函数系，那么如果能够确保扩大模型尺寸以及添加更多层参数之后其能够表达的函数系范围完整覆盖之前的一系列小尺寸模型，那么完整训练之后就能够实现模型性能的稳定提高了。

残差连接的设计就是为了实现这一点。它通过引入额外的路径，将神经网络某些层计算的输入张量直接加到输出张量上，由此就可以形成一个残差块。

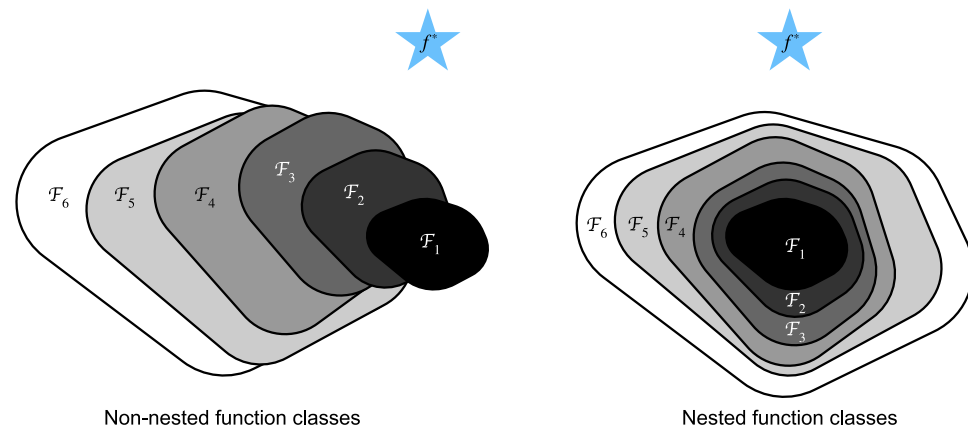


Figure 4: ResNet 灵感示意图，通过残差连接可以确保模型的表达范围稳定扩大

这就相当于输入张量跳过了神经网络的很多层计算而直接进入后续的计算流程，由此就可以确保使用各种残差块搭建起来的 ResNet 必定可以覆盖任何一个更小尺寸模型的表达能力。

ResNet 在 2015 年推出后几乎结束了 ImageNet 的比赛，其以 3.57% 的错误率直接超越了人类的平均水平，比前一年的冠军 GoogLeNet 近乎减半。后续其他人引入残差连接之后也都复现出了相当可观的性能进步，为我们今天的神经网络模型结构奠定了基础。

也正因为 ImageNet 分类任务的性能接近上限，这一赛事也最终于 2017 年正式结束，完成了其历史使命。

模型结构可以参考代码“resnet.py”，这里实现了一个 18 层的 ResNet 来实现 MNIST 分类任务，可以看到它的性能表现相当之好。

实际上此处的 ResNet 的能力已经足够胜任相当复杂的任务了，在 MNIST 数据集上多少有些大材小用，最终已经存在一些过拟合的现象。(总参数量 701,178)

Epoch [1/5],	准确率: 96.47%
Epoch [2/5],	准确率: 98.54%
Epoch [3/5],	准确率: 98.82%
Epoch [4/5],	准确率: 98.88%
Epoch [5/5],	准确率: 98.73%

input image tile

output segmentation map

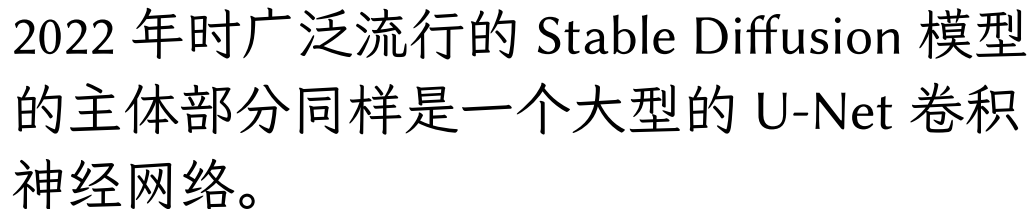
conv 3x3, ReLU

copy and crop

max pool 2x2

up-conv 2x2

conv 1x1



敬请期待下集: 模型结构-下

下集当中会主要讲解 NLP 领域最常使用的
RNN 与 Transformer 架构, 也会涉及到 ViT 以
及许多后续改良